

ENGINEERING SOFTWARE

From Reliable Code to Production Systems

Scott Brodie Forsyth

A compact, evidence-aware guide to designing, building, testing, securing, deploying, and operating software.

Engineering software is the practice of making
important behaviour dependable under changing conditions.

Copyright © 2026 Scott Brodie Forsyth.

This edition is prepared as an editable educational manuscript. Standards, dependencies, and platform interfaces change; verify operational details before using them in a live system.

Source code examples are provided for learning and are licensed for reuse with the repository. Third-party standards and books remain under their original licenses.

Contents

How to use this book	xi
1 Engineering as a way of reasoning	1
1.1 The object of the discipline	1
1.2 Claims, evidence, and recommendations	1
1.3 A system boundary is a decision	2
1.4 The feedback loop	2
1.5 A practical quality model	2
1.6 Chapter summary	3
2 Requirements, specifications, and stakeholders	5
2.1 From requests to obligations	5
2.2 Stakeholder analysis	5
2.3 Functional and quality requirements	6
2.4 Specifications as executable agreements	6
2.5 Acceptance and examples	7
2.6 Change and traceability	7
2.7 Chapter summary	7
3 Decomposition, abstraction, and boundaries	9
3.1 The purpose of decomposition	9
3.2 Cohesion and coupling	10
3.3 Abstraction and the cost of being wrong	10
3.4 Designing a boundary	10
3.5 When a boundary is too early	11
3.6 Chapter summary	11
4 APIs, data modelling, and persistence	13
4.1 An interface is a promise over time	13
4.2 Identifiers and time	13
4.3 Relational modelling	14
4.4 Transactions and consistency	14
4.5 Schema evolution	15
4.6 Chapter summary	15

5	Concurrency, networking, and distributed systems	17
5.1	Concurrency is about interleaving	17
5.2	Memory and message boundaries	17
5.3	Ordering and causality	18
5.4	Consistency is a product decision	18
5.5	Queues and workers	18
5.6	A small state machine	19
5.7	Chapter summary	19
6	Architecture styles and trade-offs	21
6.1	Architecture is a set of consequential decisions	21
6.2	Common styles	21
6.3	Architecture decision records	22
6.4	Data ownership	22
6.5	Architecture fitness functions	22
6.6	Architecture review	23
6.7	Chapter summary	23
7	Object-oriented, functional, procedural, and data-oriented design	25
7.1	Paradigms are lenses	25
7.2	Encapsulation and state	25
7.3	Functional core, imperative shell	26
7.4	Procedures and data flow	26
7.5	Data-oriented thinking	26
7.6	Choosing a representation	26
7.7	Chapter summary	27
8	Version control, review, and collaborative development	29
8.1	Version control as a reasoning tool	29
8.2	Code review	29
8.3	Review comments	30
8.4	Documentation that earns its keep	30
8.5	Branching and integration	30
8.6	Psychological safety and accountability	30
8.7	Chapter summary	31
9	Testing, analysis, debugging, and proof	33
9.1	Testing as evidence	33
9.2	Test design	33
9.3	Static analysis and types	34
9.4	Debugging as hypothesis testing	34
9.5	Formal methods in proportion	34
9.6	Mutation and negative evidence	35
9.7	Chapter summary	35

10 Reliability, resilience, scalability, and capacity	37
10.1 Define reliable for whom	37
10.2 SLOs and error budgets	37
10.3 Failure modes and resilience	37
10.4 Scalability and capacity	38
10.5 Load, stress, and performance tests	38
10.6 Backups and recovery	38
10.7 Chapter summary	39
11 Security engineering, privacy, and threat modelling	41
11.1 Security is a system property	41
11.2 Threat modelling	41
11.3 Secure defaults and input handling	42
11.4 Secrets and cryptography	42
11.5 Privacy by design	42
11.6 Supply-chain security	42
11.7 Security verification	43
11.8 Chapter summary	43
12 CI/CD, containers, cloud systems, and infrastructure as code	45
12.1 Delivery is a controlled experiment	45
12.2 Build once, configure at runtime	45
12.3 Containers	46
12.4 Infrastructure as code	46
12.5 Deployment strategies	46
12.6 CI workflow example	47
12.7 Chapter summary	47
13 Observability, operations, incidents, and postmortems	49
13.1 From running to knowing	49
13.2 Signals	49
13.3 Alerting	49
13.4 Runbooks and safe operations	50
13.5 Incident response	50
13.6 Postmortems	50
13.7 Change and observability	50
13.8 Chapter summary	51
14 Refactoring, technical debt, legacy systems, and migrations	53
14.1 Maintenance is the normal state	53
14.2 Refactoring	53
14.3 Technical debt	54
14.4 Legacy systems	54
14.5 Migrations	54
14.6 When to rewrite	54
14.7 Chapter summary	55

15 Accessibility, internationalization, ethics, sustainability, and AI assistance	57
15.1 People are part of the operating environment	57
15.2 Practical accessibility	57
15.3 Internationalization	58
15.4 Sustainability	58
15.5 Ethics and power	58
15.6 Responsible AI-assisted development	58
15.7 Inclusive design review	59
15.8 Chapter summary	59
16 Estimation, teams, incidents, and engineering management	61
16.1 Estimation under uncertainty	61
16.2 Planning and sequencing	61
16.3 Team organization	62
16.4 Engineering management	62
16.5 Incidents and learning	62
16.6 Decision rights	62
16.7 Healthy delivery pressure	62
16.8 Chapter summary	63
17 Case study: engineering Ledgerline	65
17.1 Context	65
17.2 Requirements and boundaries	65
17.3 Architecture	66
17.4 API contract	66
17.5 Reviewing the implementation	66
17.6 Failure analysis	69
17.7 What would change at larger scale	69
17.8 Chapter summary	69
18 Capstone: from requirement to operated release	71
18.1 The assignment	71
18.2 Required deliverables	71
18.3 A release checklist	71
18.4 How to present the design	72
18.5 Final perspective	72
A Repository and tool guide	73
A.1 Quick start	73
A.2 Example inventory	74
A.3 Validation policy	74
A.4 Updating the manuscript	74

B Cumulative exercises	75
B.1 Foundations	75
B.2 Design and data	75
B.3 Concurrency and architecture	75
B.4 Quality and operation	76
B.5 Security and human impact	76
B.6 Capstone extensions	76
 C Selected solutions and reasoning	 77
C.1 Foundations	77
C.2 Design and data	77
C.3 Concurrency and architecture	78
C.4 Quality and operation	78
C.5 Security and human impact	78
C.6 Capstone reasoning	79
 Glossary	 81

How to use this book

Software engineering is not a bag of rituals. It is a way of reducing the gap between what a system is supposed to do and what it actually does when people, networks, data, deadlines, attackers, and future maintainers interact with it.

Each chapter moves between four levels: a mental model, a concrete technique, the conditions under which the technique helps, and the way it can fail. The running project is a small order-intake service called *Ledgerline*. It is intentionally ordinary: it accepts orders, validates them, stores them, exposes an API, emits useful signals, and must remain understandable six months later. The repository contains smaller examples so that the ideas can be run without proprietary infrastructure.

Words such as *must*, *should*, and *may* are used carefully. When they are capitalized inside a quoted standard, they retain the meaning of that standard. Elsewhere, they indicate the strength of the author's advice, not a universal law.

Chapter 1

Engineering as a way of reasoning

1.1 The object of the discipline

Programming is the act of expressing a computation. **Software engineering** adds a harder question: how can that computation remain correct, understandable, secure, and economically maintainable while its environment changes? The environment includes users, regulations, dependencies, hardware, networks, other teams, and the people who will inherit the code without the original author's memory.

The word *engineering* is useful because it points to constrained design. An engineer makes assumptions explicit, models important failure modes, chooses materials and boundaries, measures the result, and manages uncertainty. Software has unusual properties: it is easy to copy, cheap to modify in one place, and often difficult to observe as a whole. A change that looks local can alter security, latency, data interpretation, or the behaviour of a distant client.

Quality is not a single attribute. It is the degree to which a system satisfies the requirements that matter in its context, including requirements about change, operation, safety, privacy, and future understanding.

1.2 Claims, evidence, and recommendations

Technical writing often mixes three kinds of statement. A specification defines what an implementation must do. Empirical research reports observations under specified conditions. Professional practice is accumulated experience that may be useful without being a universal law. A recommendation is a judgment about a particular context. Keeping these categories separate prevents cargo cults.

For example, HTTP semantics are defined by an IETF standard; the assertion that an HTTP method is safe or idempotent is a protocol claim that can be checked against that standard [4]. The belief that small pull requests are easier to review is a

practice hypothesis: often plausible, but dependent on change shape, reviewer skill, and system complexity. The decision to use a modular monolith for a small product is an engineering recommendation whose strength comes from the constraints, not from the label.

1.3 A system boundary is a decision

Every system description has a boundary. Inside it are behaviours the team can change directly; outside it are actors, services, devices, laws, and physical conditions. A boundary is never purely technical. It encodes ownership and responsibility.

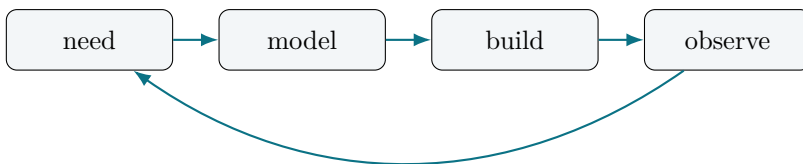
The Ledgerline boundary is deliberately modest:

- a caller submits an order containing a customer identifier and line items;
- the service validates domain invariants and returns a stable order identifier;
- a relational store records the accepted state;
- operators can tell whether the service is alive, useful, and overloaded;
- a failure does not silently create a second order when the caller retries.

The last point is not an implementation detail. It is a business property that leads to an idempotency key, a database constraint, a response contract, and a recovery test. Engineering begins when a vague concern becomes a checkable property.

1.4 The feedback loop

Good development is a loop rather than a waterfall of documents:



The model may be a test, a state machine, a schema, a sequence diagram, or a small written invariant. The point is not to predict every detail. It is to make the important assumptions cheap to challenge before they become expensive code or production incidents.

1.5 A practical quality model

Before choosing tools, ask which of these dimensions are material:

Dimension	Question to answer
Correctness	Does the system produce the required result for valid and invalid inputs?
Safety and security	What must never happen, and who might deliberately cause it?
Reliability	What happens when dependencies, data, or operators behave unexpectedly?
Operability	Can a responsible person detect, diagnose, and recover from failure?
Changeability	Can a team modify one concern without guessing about unrelated consequences?
Human fit	Can people with different abilities, languages, roles, and incentives use and govern it?

These dimensions compete. Stronger durability may cost latency; a highly generic API may be harder to understand; a fast migration may increase rollback risk. The engineering response is not to maximize every quality. It is to state the trade-off and make the consequence visible.

Exercise. Write five falsifiable properties for a service that accepts payments. For each, name the boundary at which it is measured and one test or observation that could show it is false.

1.6 Chapter summary

Software engineering is disciplined reasoning under constraints. Begin with properties rather than technologies, distinguish standards from evidence and recommendations, draw a deliberate system boundary, and use short feedback loops to turn assumptions into tests and observations.

Chapter 2

Requirements, specifications, and stakeholders

2.1 From requests to obligations

“Make checkout faster” is a request, not yet a requirement. It does not say which users, which journey, which measurement boundary, or what trade-off is acceptable. A useful requirement names an actor, an observable behaviour, a condition, and a threshold or invariant when one matters.

One Ledgerline requirement can be written as:

When a caller submits a syntactically valid order with an unused idempotency key, the service shall persist one accepted order and return its identifier. Repeating the same request with the same key shall return the original result without creating another order.

This statement creates questions: How long is a key retained? What if the first request timed out after commit? Must the payload match on reuse? What error does a conflicting payload receive? Requirements become useful when their boundary cases are discussed rather than hidden.

2.2 Stakeholder analysis

A stakeholder is anyone who can affect, use, constrain, be affected by, or be held accountable for the system. The person requesting a feature is not always the person who bears its risks. A small table is often enough to uncover this.

Stakeholder	Interest	Questions
Customer	A correct, understandable order	What counts as confirmation? Can the order be retried safely?
Support	Ability to explain outcomes	Can a person find the request, status, and reason for rejection?
Operator	Stable service and recoverable data	What is monitored, backed up, and safe to change?
Finance	Reconciled money	Which values are authoritative, and how are corrections audited?
Security and privacy	Limited exposure and accountability	Which data is collected, for what purpose, and under whose authority?

Stakeholder analysis does not mean accepting every preference. It makes the conflict explicit. A customer may want unlimited retention for convenience while privacy policy requires deletion. An operator may want a safe pause while a commercial team wants availability. Engineering supplies options and evidence; governance decides which values prevail.

2.3 Functional and quality requirements

Functional requirements describe behaviour: create an order, cancel it, export a record. Quality requirements describe conditions on that behaviour: a latency percentile, a recovery time, an accessibility criterion, or a permitted data residency. The categories interact. “Returns a result” is incomplete if the result is too late for the user, cannot be accessed by keyboard, or exposes a secret in logs.

Use a requirement record with at least:

1. identifier and short name;
2. rationale and stakeholders;
3. normative statement;
4. acceptance examples, including invalid cases;
5. measurable quality target and measurement boundary;
6. dependencies, assumptions, and unresolved decisions;
7. owner and review date.

2.4 Specifications as executable agreements

A specification is valuable when independent people can use it to predict the same behaviour. Examples include an OpenAPI document, a SQL schema, a property test, a protocol state machine, or a runbook. A prose requirement remains important for purpose and context, but a machine-checkable representation catches drift earlier.

Do not make the specification larger than the uncertainty it resolves. A stable public API deserves explicit compatibility rules. A private function may need only a

type, a test, and a clear name. Excess ceremony creates documents that are neither read nor updated.

2.5 Acceptance and examples

Examples expose ambiguity. The following set describes the same rule from three angles:

- Given two units at 1,250 cents, the subtotal is 2,500 cents.
- Given quantity zero, the request is rejected before persistence.
- Given the same idempotency key twice with the same payload, the second response refers to the first order.

Examples are not a replacement for general rules. They are probes that make the rule concrete and reveal missing decisions. Property-based tests can then state the broader invariant: for every valid collection of positive integer prices and quantities, the total is an integer and equals the sum of each product.

2.6 Change and traceability

Requirements change because the world changes. A useful traceability chain is: requirement -> design decision -> code -> automated check -> operational signal. The chain should be lightweight enough to maintain. A commit message or issue identifier can be sufficient for a small team; regulated work may need a formal record. The principle is the same: a future reader should be able to ask why a constraint exists and find evidence.

Trade-offs. Detailed upfront specifications reduce some ambiguity but can freeze wrong assumptions. Pure improvisation learns quickly but can leave invisible contracts. Use more formality where the cost of misunderstanding is high, and keep the feedback cycle short where uncertainty is high.

Exercise. Turn “the report page should feel responsive” into a measurable requirement. State the user journey, the measurement boundary, the percentile and time window, the test data shape, and one reason the target might need revision.

2.7 Chapter summary

Requirements become engineering material when they identify actors, behaviour, conditions, and evidence. Include quality properties, examine stakeholders who bear risks, use examples to expose ambiguity, and keep a trace from intent to operation without creating paperwork that nobody can maintain.

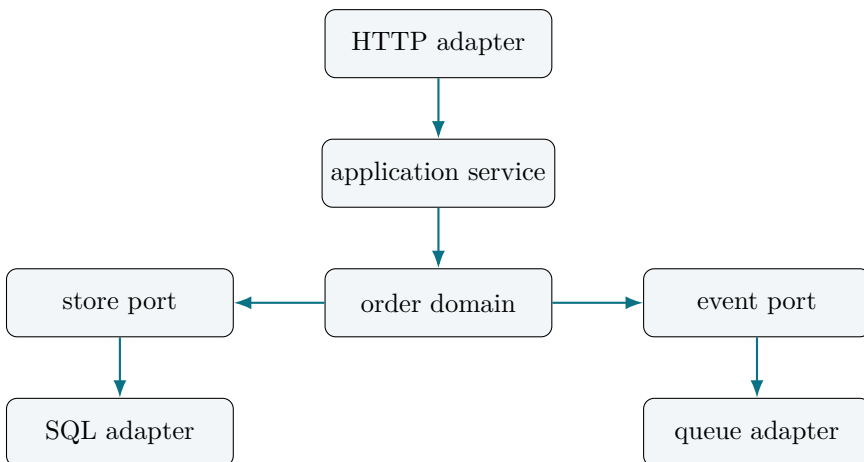
Chapter 3

Decomposition, abstraction, and boundaries

3.1 The purpose of decomposition

Decomposition turns a large problem into units that people can understand, change, and test. It is not merely splitting a file. A good boundary hides a decision that is likely to change or that should be protected by an invariant. Parnas’s classic discussion treats information hiding as a criterion for module design, rather than a preference for a particular syntax [11].

For Ledgerline, a useful first decomposition is:



The arrows represent dependency direction, not necessarily runtime network traffic. The domain does not need to know whether persistence is SQLite, PostgreSQL, or an in-memory fake. The adapter owns translation between a technology’s vocabulary and the domain’s vocabulary.

3.2 Cohesion and coupling

Cohesion asks whether the responsibilities inside a unit belong together. **Coupling** asks how much one unit knows about another. High cohesion and low unnecessary coupling are useful heuristics, not scores to maximize blindly. Some coupling is valuable: an invariant about money may be kept close to the database constraint that enforces it. The danger is accidental coupling, such as a UI importing a database row shape and thereby making a storage migration a UI change.

A dependency is especially expensive when it crosses:

- ownership boundaries, because coordination becomes necessary;
- time boundaries, because queues and retries introduce duplication or delay;
- trust boundaries, because input must be authenticated and validated;
- data-lifetime boundaries, because privacy and deletion rules differ;
- release boundaries, because two versions must coexist.

3.3 Abstraction and the cost of being wrong

An abstraction removes detail so that a person can reason about a larger unit. It also hides information. An abstraction is successful when the hidden detail can vary without surprising its clients. It is unsuccessful when the abstraction promises a simple concept but leaks the complexity through exceptions, latency, ordering, or resource usage.

The phrase “repository” can hide a database, but it cannot make a database transaction disappear. If callers assume that ‘save’ is atomic while the implementation performs three independent writes, the abstraction lies. Make important semantics explicit: `save_order_and_outbox_event` communicates more than ‘save’ if atomicity is essential.

3.4 Designing a boundary

For each proposed module, write four sentences:

1. Its responsibility is ...
2. It guarantees ...
3. It may change without requiring ...
4. It must not know ...

Then test the boundary against a plausible change. If changing the tax rate requires editing HTTP parsing, SQL queries, and every unit test, the domain rule is scattered. If a tax module cannot be tested without a network and a database, the boundary is probably too entangled.

3.5 When a boundary is too early

Premature decomposition creates indirection before the shape of the problem is understood. Five interfaces around five lines of code can make a small system harder to read. Start with seams that protect a real decision: an external API, a security check, a durable write, or a volatile policy. Refactor toward clearer boundaries as evidence accumulates. The repository's Python example keeps the domain function small because the purpose is visible; it does not manufacture a framework around it.

Exercise. Take a single function that validates and persists an order. List three changes that could occur independently. Design boundaries that isolate those changes, then name one cost introduced by each boundary.

3.6 Chapter summary

Decompose around changing decisions and protected invariants. Use information hiding, cohesion, and coupling as reasoning tools. Treat abstractions as contracts about semantics, not just shorter names, and delay indirection when it does not yet protect a real change.

Chapter 4

APIs, data modelling, and persistence

4.1 An interface is a promise over time

An **API** is more than a URL or method signature. It defines inputs, outputs, errors, identity, authorization, ordering, consistency, and compatibility. A client can depend on behaviour that was never written down, so an API should make its important promises deliberate.

For an HTTP endpoint, document at least:

- the resource and method semantics;
- accepted representations and validation rules;
- success and error status codes with stable error shapes;
- authentication, authorization, rate limits, and sensitive fields;
- retry and idempotency behaviour;
- pagination, ordering, time zones, and version-compatibility rules.

HTTP is standardized as a stateless application-level protocol with shared semantics across versions; the standard does not dictate your business resource model [4]. This distinction matters. A ‘POST /orders’ contract is yours, while the semantics of status codes and method properties are governed by the HTTP specification.

4.2 Identifiers and time

Identifiers should communicate less than they need to. An opaque identifier reduces accidental coupling to storage order and makes enumeration harder, but it does not replace authorization. If a client needs chronological ordering, provide an explicit timestamp or cursor rather than asking it to decode an ID.

Time has at least three meanings: an instant in a global timeline, a local calendar representation, and a duration measured by a monotonic clock. Store instants in an unambiguous form, carry time-zone information when presenting calendar values, and use a monotonic clock for elapsed-time measurements. Never use wall-clock time to decide whether a timeout duration has elapsed.

4.3 Relational modelling

A relational schema is an executable set of invariants. In the example schema, positive quantities are checked in the database even though application code also validates them. Defence in depth is useful because imports, migrations, administrative scripts, and future services can bypass one validation path.

The order total is stored as integer cents. Decimal money is a domain choice; the important point is that binary floating-point approximations are not silently used for a value whose legal meaning is exact. The database view recalculates the sum of line items. In a production design, decide whether the stored total is a snapshot, a cache, or an authoritative ledger value, and reconcile it explicitly.

Listing 4.1: A constraint is part of the data contract.

```
1 CREATE TABLE order_items (  
2     order_id TEXT NOT NULL REFERENCES orders(id),  
3     sku TEXT NOT NULL CHECK (length(trim(sku)) > 0),  
4     quantity INTEGER NOT NULL CHECK (quantity > 0),  
5     unit_price_cents INTEGER NOT NULL CHECK (unit_price_cents > 0)  
6     ,  
7     PRIMARY KEY (order_id, sku)  
8 );
```

4.4 Transactions and consistency

A transaction groups changes so that the database presents an allowed state at its commit boundary. The exact isolation guarantees vary by database engine and configuration. Do not infer serializable behaviour from the word “transaction.” Ask what anomalies are possible: dirty reads, lost updates, write skew, or phantoms. Then choose a constraint, lock, isolation level, or retry strategy that matches the invariant.

The outbox pattern is useful when a durable state change must cause a message. Write the domain change and an outbox row in the same local transaction. A separate publisher reads unclaimed outbox rows and sends them with at-least-once delivery. Consumers must therefore be idempotent. The pattern does not create exactly-once effects across arbitrary systems; it creates a recoverable handoff.

4.5 Schema evolution

A safe migration is compatible with the versions that may coexist during rollout. The expand-and-contract sequence is a common strategy:

1. add a nullable or additive representation;
2. deploy code that can read both and writes the new form;
3. backfill and validate existing data;
4. switch readers and enforce the new invariant;
5. remove the old representation only after rollback is no longer needed.

This costs temporary complexity. The alternative, a single destructive change, may be reasonable for a private database with a verified restore path and a maintenance window. The decision depends on data value, downtime tolerance, rollout topology, and rollback feasibility.

Exercise. Design an endpoint for retrieving a customer's orders. Specify pagination, ordering, authorization, deletion behaviour, and what happens when a client repeats a request after a timeout.

4.6 Chapter summary

APIs are long-lived behavioural contracts. Model identity, time, errors, retries, and compatibility explicitly. Use database constraints as executable invariants, understand the actual transaction guarantees, and evolve schemas in a way that respects the versions present during deployment.

Chapter 5

Concurrency, networking, and distributed systems

5.1 Concurrency is about interleaving

Concurrency means multiple activities can make progress during overlapping intervals. Parallelism is simultaneous execution. A single-threaded event loop can be concurrent without being parallel; multiple CPU cores can be parallel. The distinction matters for reasoning, resource limits, and performance claims.

The basic danger is an interleaving that violates an invariant. Suppose two requests both read an account balance of 100 and both decide that a withdrawal of 80 is allowed. If each writes 20, the final balance looks plausible but the system has accepted 160 of withdrawals. A transaction, an atomic conditional update, or a serialized command can protect the rule.

Do not ask only whether each operation is correct in isolation. Ask which interleavings are possible, which state is shared, and which invariant must hold after every visible transition.

5.2 Memory and message boundaries

In one process, threads share memory and require synchronization. Across processes, machines, or organizations, messages cross a boundary and can be delayed, duplicated, reordered, or lost. A network timeout does not tell the caller whether the remote operation failed or succeeded but its response was lost. This is the source of many duplicate writes.

A robust request path defines:

- a deadline propagated through every downstream call;
- a bounded retry policy only for operations that are safe to repeat;
- an idempotency key or deduplication record for retried effects;

- a clear failure result when the outcome is unknown;
- telemetry that distinguishes timeout, refusal, overload, and validation.

Retries multiply load during failure. Exponential backoff with jitter spreads attempts, but it is not a cure for an overloaded dependency. A caller should retry only when the error is plausibly transient and the total deadline still allows useful work. A circuit breaker can stop calls to a failing dependency, but it introduces its own state machine and false-open trade-off.

5.3 Ordering and causality

Wall-clock timestamps are not a reliable global ordering in a distributed system. Clocks differ and can move. Lamport’s work on logical clocks shows how a system can represent a partial order of events without pretending that every event has one exact global time [7]. In practice, choose the smallest ordering guarantee that the product needs: per-account sequence numbers, partition-local order, or an explicit version check.

5.4 Consistency is a product decision

An eventually consistent view can be correct for a feed whose purpose is discovery, but unacceptable for a bank balance or an authorization decision. “Strong” and “eventual” are not complete specifications. State what a reader may observe after a write, which conflicts can occur, and how a user can repair or understand them.

The CAP theorem is frequently reduced to a slogan about choosing two of three properties. The more useful interpretation is narrower: under a network partition, a distributed data system cannot simultaneously provide a particular strong consistency guarantee and uninterrupted availability for every operation. The actual design space includes latency, scope of consistency, failure model, and recovery strategy [2].

5.5 Queues and workers

Queues decouple producers and consumers in time. They absorb bursts and allow a worker pool to process work at a controlled rate. They also create backlog, duplicate delivery, poison messages, and delayed user-visible state. A queue is not a free buffer; it is a new stateful subsystem.

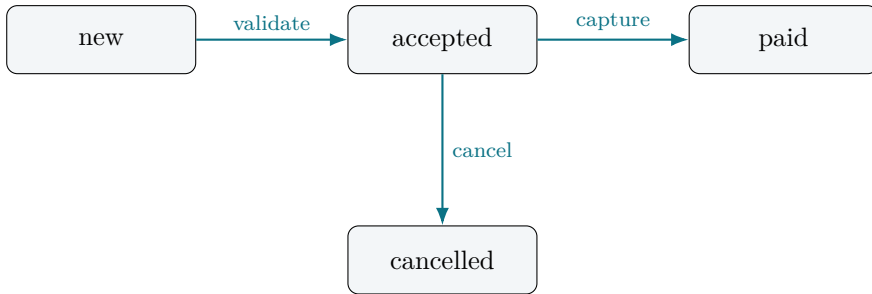
For a message handler, specify:

1. what makes a message valid;
2. the acknowledgment point;
3. the effect of a crash before and after acknowledgment;
4. maximum attempts and dead-letter handling;
5. deduplication identity and retention;

6. how operators replay, quarantine, or delete a message safely.

5.6 A small state machine

Model a payment or order lifecycle as a state machine before implementing it:



The diagram makes illegal transitions visible. It also raises operational questions: what if capture succeeds but the response times out? The answer may be a reconciliation state rather than an optimistic retry.

Exercise. For a queue consumer that sends an email, list the states before and after a crash at each of these points: before send, after send, before acknowledgment, and after acknowledgment. Choose a deduplication strategy and state its limits.

5.7 Chapter summary

Concurrency is reasoning about interleavings; distribution adds uncertain messages and partial failure. Define deadlines, retryability, idempotency, ordering, consistency, and recovery. Treat queues and state machines as explicit parts of the product rather than invisible implementation details.

Chapter 6

Architecture styles and trade-offs

6.1 Architecture is a set of consequential decisions

Architecture describes the significant structures, boundaries, and constraints that shape a system. A diagram is useful only when it helps someone decide, implement, operate, or change the system. A box labelled “service” is not an architecture unless its responsibility, communication, data ownership, and failure behaviour are understood.

6.2 Common styles

Style	Helps when	Costs and failure modes
Modular monolith	One team needs strong local transactions and simple deployment	A shared process can become tangled; scaling and isolation are coarse.
Layered system	Responsibilities have a stable vertical flow	Layers become pass-through plumbing or leak abstractions.
Ports and adapters	Domain logic should remain independent of infrastructure	Too many interfaces obscure a small system; mappings must be maintained.
Event-driven	Producers and consumers need temporal decoupling	Ordering, retries, schemas, and debugging become distributed concerns.
Microservices	Teams need independent ownership and deployment at meaningful boundaries	Network failure, data ownership, versioning, and operational cost multiply.

None is the universally mature choice. A microservice is not automatically more scalable, and a monolith is not automatically a prototype. The decision should be driven by boundaries of change, ownership, failure isolation, deployment independence, data consistency, and capacity rather than by fashion.

6.3 Architecture decision records

An architecture decision record (ADR) keeps a decision close to its reasons:

1. Context: what forces and constraints exist?
2. Decision: what will we do?
3. Alternatives: what serious options were considered?
4. Consequences: what becomes easier, harder, or newly risky?
5. Status: proposed, accepted, superseded, or rejected.

An ADR is not a vote and not a substitute for measurement. It is a memory aid. If a decision was made under an assumption about traffic, team size, or a vendor, record the assumption and a trigger for revisiting it.

6.4 Data ownership

In a distributed architecture, the hardest boundary is often data. If two services write the same tables, ownership is nominal. If they have separate stores, cross-service transactions become workflows with compensation or reconciliation. The right choice depends on consistency requirements and failure tolerance.

For Ledgerline, keep order state in one database while the domain is small. A separate payment service may make sense when legal responsibility, risk controls, or independent scaling require it. Splitting it merely to create more boxes would make failures harder to recover from without solving a real constraint.

6.5 Architecture fitness functions

A **fitness function** is a repeatable check for an architectural property. Examples:

- a dependency rule that prevents the domain package importing HTTP code;
- a migration test that proves an old and new application version can coexist;
- a load test that measures p95 latency at a stated arrival rate;
- a security test that rejects access across a tenant boundary;
- an accessibility test that checks keyboard navigation for a critical flow.

Fitness functions do not prove the architecture is good. They catch regression against properties the team has decided matter. A metric without a decision attached can become decorative or actively misleading.

6.6 Architecture review

Review the smallest set of questions that could change the design:

- What are the critical user journeys and failure consequences?
- Which state is authoritative and which copies are disposable?
- What happens when every dependency is slow, unavailable, or inconsistent?
- How is a change released and rolled back while old clients remain?
- Which trust boundaries, secrets, personal data, and audit obligations exist?

Trade-offs. Centralization simplifies coordination and can create a bottleneck. Distribution can improve isolation and team autonomy and can also multiply failure modes. The correct architecture is the smallest one that meets the real constraints with known consequences.

Exercise. Choose a modular monolith, a service split, or an event-driven design for an application that has one team, 20 requests per second, a strict audit trail, and one database. Defend the choice, then identify the first signal that would make you reconsider it.

6.7 Chapter summary

Architecture is about boundaries and consequences. Select a style for ownership, failure, change, data, and capacity reasons. Record assumptions in ADRs and turn important architectural properties into automated or observable fitness checks.

Chapter 7

Object-oriented, functional, procedural, and data-oriented design

7.1 Paradigms are lenses

A programming paradigm emphasizes certain ways of representing state, behaviour, and composition. Object-oriented design groups data with operations and uses encapsulation and polymorphism. Functional design emphasizes expressions, immutable values, and explicit transformation. Procedural design emphasizes ordered commands and control flow. Data-oriented design arranges data for the access patterns of the hardware and workload.

Real systems combine them. A functional core can sit inside an object-oriented application; a procedural migration script can protect a data-oriented storage layout. The question is whether the chosen representation makes the important invariants and costs visible.

7.2 Encapsulation and state

Encapsulation means clients depend on a stable set of operations rather than directly manipulating representation. It helps when the invariant is local and the state has a meaningful lifecycle. It becomes harmful when a class is merely a namespace for unrelated functions or when inheritance creates hidden coupling.

Prefer composition when behaviour varies independently. Inheritance can express a genuine substitutability relation, but sharing code is not by itself a reason to create a subtype. A payment gateway that supports ‘authorize’ and ‘capture’ may be a port with several adapters; it need not form a deep class hierarchy.

7.3 Functional core, imperative shell

Pure functions are easier to test because their result depends on explicit input. Side effects are still necessary for useful software. A practical split is:

- imperative shell: read HTTP, clock, randomness, database, and network;
- functional core: validate, calculate totals, decide transitions, and construct commands from explicit values.

The Python `total_cents` example follows this approach. It accepts data, applies rules, and has no network or database dependency. The shell decides when and how to call it. This does not eliminate integration tests; it makes the rule-level tests fast and diagnostic.

7.4 Procedures and data flow

Procedural code can be the clearest form for a workflow. A migration or a shell script is often easier to audit as a sequence of explicit steps than as a graph of objects. The risk is accidental shared state and unclear error handling. Keep preconditions, postconditions, and cleanup paths visible.

7.5 Data-oriented thinking

Performance depends on the whole path: algorithm, data layout, allocation, cache behaviour, I/O, concurrency, and workload shape. Data-oriented design starts with the hot operations and organizes data so they do not repeatedly load irrelevant fields. It is useful in simulations, games, analytics, and high-throughput infrastructure; it can make ordinary business code less readable when applied without a measured bottleneck.

7.6 Choosing a representation

Use these questions:

1. What state must be protected by an invariant?
2. Which effects are unavoidable, and can they be isolated?
3. Does behaviour vary by a stable capability or by a changing data value?
4. Is the workload limited by CPU, memory, allocation, I/O, or coordination?
5. What will a future reader need to change safely?

Situation	Often useful starting point
Local lifecycle in-variant	Encapsulation around a small state machine
Many transformations and rules	Pure functions and immutable values
Explicit workflow or migration	Procedural sequence with checked steps
Measured hot path and predictable data	Data-oriented layout and profiling

Exercise. Represent an order cancellation in two ways: a mutable object and a pure transition function. For each, state where invalid state is prevented and where side effects occur. Which representation gives the clearer test failure?

7.7 Chapter summary

Paradigms are tools for making state, effects, and cost visible. Combine them deliberately. Prefer the simplest representation that protects the important invariants and fits the measured workload, and avoid turning stylistic taste into an architectural law.

Chapter 8

Version control, review, and collaborative development

8.1 Version control as a reasoning tool

Version control records a sequence of states and the intent behind changes. Git is a distributed revision control system; its commands let a team compare, select, combine, and recover work. The commands are not the process. A process is good when it makes change reviewable, integrates often enough to expose conflicts, and preserves a trustworthy path from request to released state.

A useful commit is coherent: one idea, a message that explains why, and tests that support the changed behaviour. Small does not mean trivial. A database migration and the code that requires it may belong in one change if separating them would create an invalid intermediate state.

8.2 Code review

Review is a risk-reduction activity, not a search for stylistic agreement. Read the requirement, then the design, then the diff and its tests. Ask:

- Is the behaviour correct for valid, invalid, and repeated inputs?
- Does the change preserve data, security, compatibility, and performance properties that matter?
- Are failures observable and recoverable?
- Is the new abstraction justified by a real boundary?
- Can a future maintainer understand the reason and remove obsolete code?

The Git project's own review guidance emphasizes correctness, clarity, and the importance of reviewing test coverage, while also noting that its advice is not binding for every project [6]. This is a good example of a professional recommendation rather than a law.

8.3 Review comments

Prefer comments that identify risk and invite a decision:

The retry happens after the database write. If the response is lost, the caller can submit again and create a second order. Can we store the idempotency key with the result, or document why duplicate creation is acceptable here?

This is better than “please refactor.” It connects a concrete path to a property. Distinguish blocking correctness or security issues from optional suggestions. Resolve disagreements by testing assumptions or asking the owner of the requirement to decide, not by escalating the loudest preference.

8.4 Documentation that earns its keep

Documentation serves different readers:

Document	Primary question
README	How do I run, test, and orient myself?
API reference	What can I call, and what happens?
ADR	Why is the architecture this way?
Runbook	What should an operator do in a known situation?
Comment	Why is this local code non-obvious?

Keep a document near the code it describes, automate examples where possible, and delete claims that no longer hold. A short, verified runbook is more useful than a comprehensive page of guesses.

8.5 Branching and integration

Long-lived branches defer integration risk. Short-lived branches and continuous integration expose conflicts while the author still remembers the change. This does not require one branching model. A team may use trunk-based development, release branches, or a hybrid when release governance demands it. What matters is that the model matches deployment cadence and that it is possible to test the state being reviewed.

8.6 Psychological safety and accountability

Review quality depends on the social system. People need permission to report a risk, question a requirement, and admit uncertainty. Psychological safety does not mean low standards; it makes high standards discussable. Accountability is stronger when the system records decisions and evidence instead of assigning blame for every defect.

Exercise. Write three review comments on a hypothetical change that adds a cache. One should concern correctness, one privacy or security, and one operability. Mark which are blocking and why.

8.7 Chapter summary

Use version control to preserve intent and reviewable history. Review behaviour, risk, tests, and operational consequences rather than personal style. Keep documentation purpose-specific, integrate frequently enough to expose risk, and build a social environment where uncertainty can be reported early.

Chapter 9

Testing, analysis, debugging, and proof

9.1 Testing as evidence

A test is an experiment with a programmed oracle. It does not prove that all possible behaviour is correct. It provides evidence about a stated input space, environment, and observation. The strongest test suite is not the one with the largest line count; it is the one that catches important failures and explains them quickly.

Level	Best at finding	Main cost
Unit	Local rule and edge-case defects	Can miss wiring, configuration, and real dependencies
Component	Contract and integration defects	More setup and slower diagnostics
End to end	Broken user journeys and deployment assumptions	Slow, brittle, and often hard to localize
Property or model	Broad invariant violations and state-machine bugs	Requires a good model and failure shrinking

The levels are not a pyramid law. A small system with a strong component test may need few end-to-end tests. A safety-critical subsystem may need formal analysis. Choose the mix from failure cost, change rate, and observability.

9.2 Test design

For each behaviour, identify:

1. a representative valid input;
2. boundary values and malformed input;

3. a state transition or dependency failure;
4. the observable result and any invariant that must remain true.

Avoid asserting incidental representation. If a client only needs an order ID and status, do not make it depend on database column order or an internal class. Tests are also consumers of an interface; over-specified tests make refactoring expensive without increasing confidence.

9.3 Static analysis and types

Static analysis checks source without executing every path. A type checker can catch incompatible values; a linter can catch suspicious constructs; a dataflow tool can identify possible taint or resource leaks. False positives impose cost, so tune rules and provide an explicit escape hatch with a reason. A passing static check is evidence, not a proof of correctness.

Types are especially useful when they encode meaningful distinctions. A string for a customer ID and a string for a password are technically interchangeable but not semantically interchangeable. A wrapper type or validation boundary can reduce accidental mixing. Do not create types that obscure a simple domain rule.

9.4 Debugging as hypothesis testing

Debugging is a search through competing explanations. Start with a reproducible failure and preserve the original evidence. Write the expected and actual behaviour, then reduce the input or environment while keeping the failure.

A practical loop is:

1. observe: capture logs, traces, inputs, versions, and timing;
2. hypothesize: name a mechanism that could explain the observation;
3. predict: identify what else should be true if the hypothesis is correct;
4. test: run the smallest discriminating experiment;
5. change: fix the cause or update the model, then add a regression check.

Changing several things at once destroys information. The exception is an active incident where stabilization is more important than diagnosis; record temporary changes so they can be revisited.

9.5 Formal methods in proportion

Formal methods use mathematical models to specify or analyze systems. They range from assertions and contracts to model checking, theorem proving, and formally verified compilation. The benefit is strongest when a small, critical state space can be modeled precisely. The cost is building and maintaining the model and connecting it to implementation.

An invariant can be simple:

$$\forall i \in \text{order items} : i.\text{quantity} > 0 \wedge i.\text{unit_price_cents} > 0.$$

That statement can be enforced by application validation, a database constraint, and tests. More advanced tools can explore concurrent transitions. Use formal analysis where it reduces a material risk, not as decoration.

9.6 Mutation and negative evidence

Mutation testing changes code in small ways and asks whether the tests fail. A surviving mutation points to a missing assertion, but mutation scores are not a universal quality metric. Similarly, a security test that finds no vulnerability does not establish security; it establishes only that the tested attack and observation did not reveal one.

Exercise. Create a test matrix for an order endpoint with dimensions of authentication, payload validity, duplicate key, database failure, and client timeout. Choose the smallest set of tests that covers each important interaction and explain what is left untested.

9.7 Chapter summary

Tests provide scoped evidence. Combine unit, component, journey, property, and static checks according to risk. Debug by preserving evidence and testing hypotheses. Use types, assertions, and formal models when they clarify and reduce important risk rather than because a technique is fashionable.

Chapter 10

Reliability, resilience, scalability, and capacity

10.1 Define reliable for whom

Reliability is not “the server did not crash.” It is the probability that a system performs its required function for a stated user, time, and environment. An API can be available while returning stale or incorrect data; a batch job can be correct while missing its deadline. Define the service indicator before choosing a target.

Useful indicators include successful request ratio, freshness, correctness rate, queue age, and latency percentiles. The percentile must include a population, window, and boundary. “p95 under 200 ms” is incomplete unless it says which requests, which route, whether retries are included, and where time starts and ends.

10.2 SLOs and error budgets

A service-level objective (SLO) is a target such as “99.9 percent of eligible requests in a rolling 30-day window succeed within the defined latency bound.” The remaining error budget is the permitted failure fraction. It is a decision tool: when the budget is healthy, a team may accept a risky improvement; when it is exhausted, stability work has priority. Google describes SLOs as a way to state and measure desired reliability [1].

The arithmetic is simple. For a 30-day window, 99.9 percent availability allows about 43.2 minutes of unavailable time if the measurement is continuous. The number is not a promise: maintenance, exclusions, sampling, and user impact change the interpretation. State the measurement method.

10.3 Failure modes and resilience

Resilience is the ability to continue useful operation or recover after failure. List failure modes by layer:

- input: malformed, oversized, duplicated, or adversarial;
- process: crash, memory exhaustion, deadlock, resource leak;
- dependency: timeout, partial response, incompatible version;
- data: corruption, lag, missing migration, unrecoverable deletion;
- human and organizational: bad release, unclear ownership, unavailable expert.

For each, choose a response: prevent, detect, degrade, retry, isolate, fail closed, fail open, or recover. “Fail gracefully” is not a response until the degraded behaviour is named.

10.4 Scalability and capacity

Scalability is how resource use and performance change as workload grows. It is not synonymous with speed. Capacity planning starts with a workload model: arrival rate, request mix, payload size, concurrency, data growth, and burst shape. Measure service time and a bottleneck resource. Little’s Law gives a useful relationship for a stable queueing system:

$$L = \lambda W,$$

where L is average work in the system, λ is average arrival rate, and W is average time in the system. The relationship does not by itself predict tail latency or behaviour during overload, but it exposes inconsistent capacity claims.

10.5 Load, stress, and performance tests

A performance test is only meaningful with a stated workload, environment, warm-up, data shape, concurrency, and measurement. A benchmark that reports average latency may hide a failing tail. A stress test beyond capacity can reveal recovery behaviour, but it should not be run against a production dependency without explicit authorization and safeguards.

Optimization should follow a measured bottleneck. Cache design changes correctness and invalidation; parallelism changes ordering and contention; a database index changes write cost and storage. Record the baseline, the change, and the new workload result.

10.6 Backups and recovery

A backup that has never been restored is an untested hope. Define recovery point objective (RPO) and recovery time objective (RTO), encrypt backups, control access, test restoration, and document dependencies such as keys and schema versions. A replication system can copy corruption or accidental deletion, so it is not a substitute for historical recovery points.

Exercise. Write an SLO for an order-creation endpoint, calculate its monthly error budget, and name three events that should consume it. Then specify an RPO and RTO for orders and justify them in terms of user and business impact.

10.7 Chapter summary

Reliability starts with a defined user-visible indicator. Use SLOs to make trade-offs explicit, model failure modes, plan capacity from workload, test performance with reproducible conditions, and treat restoration as a capability that must be exercised.

Chapter 11

Security engineering, privacy, and threat modelling

11.1 Security is a system property

Security engineering protects assets against unwanted actions under a stated threat model. It includes confidentiality, integrity, availability, authenticity, accountability, and privacy. A library patch cannot compensate for an endpoint that authorizes the wrong tenant or logs credentials.

Start by naming assets and consequences:

Asset	Adversary or accident	Consequence
Order data	Cross-tenant read or export error	Confidentiality and legal exposure
Payment state	Forged callback or re-play	Financial loss and reconciliation failure
Credentials	Log leak or phishing	Account takeover
Deployment pipeline	Malicious dependency or stolen token	System-wide compromise

11.2 Threat modelling

A lightweight threat model records actors, assets, entry points, trust boundaries, and abuse cases. STRIDE is one categorization technique; attack trees, misuse cases, and data-flow diagrams are alternatives. The technique is useful when it changes a design or test. Listing threats without owners or mitigations is a worksheet, not security engineering.

For an order API, ask:

1. Can an unauthenticated caller reach it?
2. How is tenant ownership derived and checked on every read and write?

3. Can a caller replay a payment callback or idempotency key?
4. Are quantities, IDs, URLs, files, and error messages bounded and encoded?
5. Which secrets cross the process, logs, build system, and backups?

11.3 Secure defaults and input handling

Default-deny authorization, least privilege, parameterized database queries, context-appropriate output encoding, safe error messages, bounded input, and secure transport are durable principles. “Validate input” is incomplete: validate syntax at the boundary, enforce domain rules in the domain, and enforce critical invariants in storage. Canonicalize before comparing where multiple encodings could represent the same resource.

Authentication identifies a principal; authorization decides whether that principal may perform a specific action on a specific resource. An authenticated user is not automatically authorized to read every order. Check authorization server-side, close to the resource access, and test both allowed and denied paths.

The OWASP ASVS provides a versioned basis for testing web application security controls and requirements; its identifiers can change between versions, so cite the exact version when using a requirement [10].

11.4 Secrets and cryptography

Do not store secrets in source control, client bundles, ordinary logs, or error messages. Use a secret manager or protected environment injection, rotate keys, limit access, and record a plan for compromise. Choose well-reviewed cryptographic primitives through a maintained library. Do not invent encryption or password hashing schemes. Encryption does not solve authorization, key management, or metadata leakage.

11.5 Privacy by design

Privacy includes purpose limitation, data minimization, access control, retention, deletion, transparency, and accountability. A personal-data field should have a reason, an owner, a retention rule, and an access path. Test exports and deletion across caches, replicas, logs, backups, analytics, and derived data. Legal obligations vary by jurisdiction; this chapter offers engineering controls, not legal advice.

11.6 Supply-chain security

Record dependency versions and integrity, review transitive risk, protect build credentials, produce provenance where possible, and make the release artifact reproducible enough to investigate. NIST’s SSDF provides a high-level vocabulary for integrating secure development practices into a lifecycle [12]. As of this edition’s research date, NIST had also published an initial public draft for a revision; a draft should not be represented as a final standard [8].

11.7 Security verification

Use layered evidence: code review, static checks, dependency review, unit and integration tests, dynamic testing, configuration checks, secret scanning, and authorized penetration testing. Keep a vulnerability response path with severity, owner, affected versions, remediation, disclosure, and verification. No checklist proves absence of vulnerabilities. The aim is to reduce likely failure, limit impact, and improve recovery.

Exercise. Draw a data-flow diagram for a multi-tenant order API. Mark trust boundaries and write five abuse cases. For each, choose one preventive control and one test that would detect a regression.

11.8 Chapter summary

Security is a property of architecture, code, data, people, and operations. Model assets and adversaries, separate authentication from authorization, use secure defaults and layered validation, minimize and govern personal data, and keep standards versioned and evidence-based.

Chapter 12

CI/CD, containers, cloud systems, and infrastructure as code

12.1 Delivery is a controlled experiment

Continuous integration (CI) makes changes build and test together. Continuous delivery keeps a releasable artifact available; continuous deployment may automatically release it after required checks. These are related practices, not synonyms. The useful question is how quickly and safely the team can move from a reviewed change to an observable, reversible production state.

A pipeline should answer:

1. What exact source revision produced this artifact?
2. Which tests, analyses, and security checks ran?
3. Which configuration is injected at deployment rather than baked into the artifact?
4. How is the artifact promoted, observed, rolled back, or rolled forward?
5. Who or what is authorized to perform each transition?

12.2 Build once, configure at runtime

Build a versioned artifact once and promote that artifact through environments. Rebuilding separately for staging and production can produce different bytes and invalidate test evidence. Configuration such as endpoints and feature flags may vary; code and dependencies should not silently vary.

Pin dependencies and base images to a reviewed policy. “Latest” is convenient for a tutorial and dangerous for reproducibility. Updates still need to happen; the point is to make them explicit and observable.

12.3 Containers

A container packages a process and its user-space dependencies while sharing the host kernel. It is not a complete security boundary by itself. Use a minimal image, run as a non-root user, drop unnecessary capabilities, make the filesystem read-only where feasible, bound resources, and avoid embedding secrets. The example Compose file demonstrates several of these controls.

Docker’s current documentation describes the Compose Specification as the recommended format for defining services, networks, and volumes; the format is implemented by modern Compose CLI versions [3]. Commands and implementation details remain version-dependent, so CI should verify the toolchain it expects.

12.4 Infrastructure as code

Infrastructure as code makes desired infrastructure reviewable and repeatable. It does not make infrastructure deterministic automatically. Providers have eventual consistency, external state, secrets, quotas, and destructive changes.

Treat infrastructure changes like application changes:

- plan and review the diff;
- separate creation, migration, and deletion risk;
- protect state and limit its sensitive contents;
- test modules in a disposable environment;
- document manual recovery when automation is unavailable.

12.5 Deployment strategies

Strategy	Benefit	Risk or requirement
Rolling	Gradual replacement with one pool	Old and new versions must coexist
Blue-green	Fast switch between two environments	Doubles capacity and complicates state
Canary	Evidence from a small traffic slice	Requires good segmentation and rollback signals
Feature flag	Decouples code release from user exposure	Flag lifecycle and combinatorial states

Rollback is not always safe. A database migration, irreversible message, or external side effect can make old code unable to run. Design forward-compatible changes and a compensating action where possible.

12.6 CI workflow example

The repository workflow uses pinned major action versions, a fresh checkout, separate example and PDF jobs, and an uploaded artifact. In a real project, add permissions with least privilege, dependency review, test result retention, protected environments, and secret handling appropriate to the platform.

Exercise. Design a release pipeline for a service with a schema migration and a queue consumer. Mark which steps can be automatic, which require approval, and what signal triggers rollback or a pause.

12.7 Chapter summary

Delivery should produce traceable, reproducible, observable transitions. Build once, configure deliberately, harden containers, review infrastructure changes, and choose deployment strategies based on compatibility and rollback reality.

Chapter 13

Observability, operations, incidents, and postmortems

13.1 From running to knowing

A running process is not necessarily a useful service. Operations asks whether the service is delivering its intended outcome and what a responsible person can do when it is not. Observability is the ability to infer internal state from externally emitted signals. OpenTelemetry defines a vendor-neutral set of telemetry concepts and uses normative terms such as **MUST** and **SHOULD** inside its specification [9].

13.2 Signals

Metrics summarize values over time: request count, error count, latency, queue age, saturation. **Logs** record discrete events with context. **Traces** connect work across process and network boundaries. Use each for the question it answers. A trace is not a replacement for a durable audit log, and a metric cannot show every cause.

Every signal needs a schema and a cost model. Include request or trace identity, route, outcome, dependency, and version where useful. Avoid high-cardinality labels such as raw user IDs in metrics. Redact secrets and personal data before emission, not only in a dashboard.

13.3 Alerting

An alert should indicate an actionable condition, have an owner, and explain the first safe action. Alert on symptoms that matter to users or operators: an SLO burn rate, queue age beyond a business limit, or inability to accept an order. Avoid alerts that merely say a process is busy unless somebody can act on them.

A dashboard is a decision surface. Arrange it from outcome to cause: service health, user journey, traffic, errors, latency, saturation, dependencies, and recent

changes. Test it during a failure; a beautiful dashboard that cannot answer “what changed?” is not operationally useful.

13.4 Runbooks and safe operations

A runbook should say when it applies, what evidence to collect, what actions are safe, what actions are irreversible, how to escalate, and how to verify recovery. Prefer reversible controls: reduce traffic, disable a feature, pause a consumer, or increase capacity. Administrative endpoints need authentication, authorization, audit, and a break-glass procedure.

13.5 Incident response

During an incident, stabilize first. Establish a coordinator, a communications channel, an incident timeline, and an explicit hypothesis. Separate observations from guesses. Record commands and configuration changes. Avoid risky experiments on a system whose state is not understood.

An incident is a period where the system’s expected behaviour is not being met; it is not limited to outages. A privacy exposure, incorrect financial result, or unreliable deployment can be an incident even when servers respond normally.

13.6 Postmortems

A useful postmortem is blameless about individual intent and precise about system conditions. Include impact, detection, timeline, contributing factors, what worked, what failed, and actions with owners and dates. Do not produce only a long list of “be more careful.” Improve a control: add a validation, limit blast radius, make a signal actionable, rehearse recovery, or change an approval path.

13.7 Change and observability

Observability must evolve with the system. A new queue requires queue age and dead-letter signals. A new authorization boundary requires denied-access audit events. A new batch job needs freshness and completeness, not only process uptime. Treat telemetry schemas as compatibility-sensitive APIs.

Exercise. Design an alert for an order service whose success SLO is burning budget rapidly. Name the indicator, aggregation window, owner, first action, and one false positive. Then write the minimum runbook needed to pause new intake safely.

13.8 Chapter summary

Operations is part of the product. Emit useful, privacy-aware signals; alert on actionable user impact; maintain reversible runbooks; coordinate incidents with evidence; and convert postmortem learning into stronger system controls.

Chapter 14

Refactoring, technical debt, legacy systems, and migrations

14.1 Maintenance is the normal state

Most software cost occurs after the first release: bugs are fixed, dependencies change, requirements grow, and people learn what the system really does. A design that is easy to create but hard to change is not finished engineering.

14.2 Refactoring

Refactoring changes internal structure while preserving specified external behaviour. Fowler's book popularized a catalog of small transformations and the discipline of keeping the system working while its design improves [5]. The key precondition is a trustworthy safety net: tests, assertions, domain invariants, or a controlled comparison.

A safe sequence is:

1. characterize current behaviour, including surprising behaviour;
2. make one structural change;
3. run fast checks and inspect the diff;
4. preserve or improve the boundary;
5. repeat, then remove the obsolete path.

Do not call a behaviour change a refactoring. A combined structural and product change makes failures harder to attribute and rollback more difficult.

14.3 Technical debt

Technical debt is a metaphor for an intentional or accidental future cost. It can be rational: a temporary adapter may enable a time-critical launch. It is dangerous when the cost is not recorded, the interest is underestimated, or no repayment trigger exists. Distinguish debt from mere code you dislike. A messy but stable module may be cheaper than a fashionable rewrite.

Record debt with:

- the shortcut and why it was taken;
- the risk or recurring cost;
- evidence of impact;
- the condition that makes repayment worthwhile;
- an owner or review date.

14.4 Legacy systems

Legacy means important software whose current behaviour is poorly understood or whose constraints are hard to change. The first task is not to replace it. It is to map dependencies, data, users, operational procedures, and hidden contracts. Add characterization tests at the boundary. Capture current outputs before changing internals.

A strangler approach can route one capability through a new implementation while the old system remains. It reduces a big-bang cutover but creates two systems, data synchronization, and ambiguous ownership during the transition.

14.5 Migrations

For a data migration, define source and target invariants, transformation rules, handling of invalid rows, reconciliation totals, rollback or compensation, and the cutover criterion. Run a dry run against representative data. Measure not only whether rows copied but whether user-visible meaning was preserved.

For a service migration, use shadow traffic only when privacy and side effects are controlled. Compare outputs with a defined equivalence relation. A new implementation may be “more correct” and still break clients that depended on old rounding, ordering, or error codes; decide which compatibility is required.

14.6 When to rewrite

A rewrite is justified by a concrete constraint: unsupported platform, unsafe security posture, impossible performance target, or cost that exceeds a staged transition. Estimate the work of reproducing undocumented behaviour, migrating data, retraining operators, and running both systems. “The code is old” is not a sufficient business case.

Exercise. Plan a migration from a legacy order table that stores money as floating point to integer cents. Include discovery, rounding policy, dual read/write period, reconciliation, rollback, and the point at which the old column can be removed.

14.7 Chapter summary

Maintenance is the main lifecycle, not a cleanup phase. Refactor in small safe steps, record technical debt as a decision with a trigger, characterize legacy behaviour, and migrate data with explicit invariants and reconciliation.

Chapter 15

Accessibility, internationalization, ethics, sustainability, and AI assistance

15.1 People are part of the operating environment

Software affects people with different abilities, languages, devices, resources, and power. Human requirements are not decorative polish. A service that cannot be used with a keyboard, that assumes one date format, or that quietly denies a group access has a functional defect for those users.

WCAG 2.2 organizes accessibility guidance around perceivable, operable, understandable, and robust principles, with testable success criteria [13]. Treat conformance as a useful baseline, not as proof that a particular person can complete a task. Test with assistive technologies and with people who use them when possible.

15.2 Practical accessibility

For a web interface, check semantic structure, heading order, labels, focus visibility, keyboard operation, contrast, motion preferences, error recovery, live-region announcements, and responsive zoom. Do not rely on color alone. Make the primary task possible without a mouse and without a fast connection.

Accessibility must include operational interfaces. A dashboard with a tiny red-green status indicator can exclude operators with color-vision differences and can encourage unsafe decisions under pressure.

15.3 Internationalization

Separate translatable content from code, support plural rules, keep user locale and time zone explicit, format numbers and dates according to context, and avoid assuming names, addresses, or writing direction. Store canonical instants and localize presentation. Test long strings, right-to-left layouts, non-ASCII input, and culturally unfamiliar calendars where relevant.

15.4 Sustainability

Software consumes energy through computation, storage, networks, manufacturing, and replacement cycles. A smaller carbon footprint can come from efficient algorithms, right-sized infrastructure, caching, data retention limits, longer hardware life, and avoiding work that has no user value. Optimization may trade energy against latency or availability; measure the relevant system boundary instead of claiming that one technique is always greener.

15.5 Ethics and power

Ask who benefits, who bears risk, who can contest a decision, and who can leave the system. A model may be statistically accurate and still unfairly allocate opportunity. A monitoring feature may improve security and also enable coercion. Document purpose, limits, affected groups, appeal paths, and governance. Ethical analysis is not solved by adding the word “responsible” to a project name.

15.6 Responsible AI-assisted development

AI assistants can accelerate drafting, search, and exploration. They can also invent APIs, reproduce insecure code, leak confidential context, or create an illusion of review. Treat generated code as an untrusted contribution:

1. specify the behaviour and constraints before prompting;
2. review generated code like an external patch;
3. run tests, static analysis, dependency and secret checks;
4. verify licenses, provenance, privacy, and security-sensitive logic;
5. retain human ownership for design, approval, and incident response.

Never accept a citation, benchmark, command, or library interface merely because it sounds plausible. Check primary documentation and execute safe examples. AI assistance is a tool for increasing feedback speed, not a transfer of accountability.

15.7 Inclusive design review

Include affected perspectives before implementation, not only in a final audit. Use a short risk register with severity, evidence, mitigation, owner, and review date. Ask what happens to a person who has no high-end device, cannot hear an alert, has a different name format, or is wrongly classified. These questions often reveal ordinary usability and reliability improvements as well as ethical risks.

Exercise. Choose a feature flag that automatically limits accounts after suspicious activity. Identify accessibility, language, privacy, fairness, and appeal risks. Design a human-review path and three metrics that could reveal harm.

15.8 Chapter summary

Design for real variation in people and contexts. Use accessibility standards as a baseline, make locale assumptions explicit, measure sustainability trade-offs, analyze power and contestability, and verify AI-assisted work with the same or stronger discipline as human-authored code.

Chapter 16

Estimation, teams, incidents, and engineering management

16.1 Estimation under uncertainty

An estimate is a prediction with assumptions, not a promise extracted from a person. Separate three quantities: effort, elapsed time, and confidence. A task may require two days of work but a week of elapsed time because it depends on an approval or a test environment.

Use ranges when uncertainty is material. State the reference class, known work, unknowns, dependencies, and what would change the estimate. Decompose until a piece is small enough to observe progress, but do not confuse more detailed fiction with more accuracy.

16.2 Planning and sequencing

Sequence work by risk and learning, not only by apparent feature value. Resolve unknown integrations early. Build a vertical slice that exercises the real path from input to persistence and observation. This uncovers hidden work sooner than building isolated layers that cannot yet run.

A useful plan has:

- outcome and non-goals;
- decision log and assumptions;
- milestones that produce observable evidence;
- dependencies and owners;
- quality and operational exit criteria;
- a change and rollback path.

16.3 Team organization

Team structure changes communication paths and ownership. Conway's law is a useful observation about correspondence between communication structures and system design, but it is not a deterministic theorem. Align ownership with cohesive capabilities where possible. Make interfaces and escalation paths explicit when ownership must cross teams.

Avoid "one person owns quality." Every engineer affects quality, while specific roles may coordinate standards, testing, security, or operations. A team is responsible for the system it changes; a platform or specialist team should reduce friction, not become a queue for every decision.

16.4 Engineering management

Management is system design applied to people, information, incentives, and constraints. Good managers make priorities, decision rights, and trade-offs clear; protect focus; develop capability; and ensure that uncomfortable evidence can travel upward. Metrics should support learning rather than invite gaming.

Useful signals include lead time, change failure rate, recovery time, defect escape, review wait, and user outcomes, but each can be misinterpreted. DORA-style delivery measures are indicators, not a scoreboard for comparing unlike teams. Combine quantitative signals with qualitative context.

16.5 Incidents and learning

The same skills used in operational incidents apply to project incidents: surface facts, assign a coordinator, stabilize, communicate, and record decisions. A missed migration, privacy concern, or repeated failed release deserves a causal review. A postmortem action should improve the system, workflow, or decision boundary; it should not merely ask people to remember harder.

16.6 Decision rights

For a recurring decision, name who recommends, who decides, who must be consulted, and who is informed. Centralize decisions that need consistency or rare expertise; delegate reversible local decisions. Ambiguous authority creates delay and makes risk invisible.

16.7 Healthy delivery pressure

Urgency is real, but speed without a definition of done is borrowing from the future. When a deadline forces a shortcut, record what is deferred, what risk it creates, and the condition for repayment. A safe temporary reduction in scope is usually easier to recover from than an invisible reduction in reliability or security.

Exercise. Create a one-page plan for a four-person team delivering a new order export. Include outcome, non-goals, risks, milestones, quality gates, owner boundaries, and a rollback plan. Include one decision that should be delegated and one that should be escalated.

16.8 Chapter summary

Estimate with ranges and assumptions. Sequence for learning and risk reduction, align ownership with capabilities, use metrics as evidence rather than targets, and turn incidents and shortcuts into explicit system improvements.

Chapter 17

Case study: engineering Ledgerline

17.1 Context

Ledgerline accepts small orders from a web client and an internal support tool. The first release has one team and modest traffic. Orders are commercially important, must be auditable, and must not be duplicated by a retry. The team chooses a modular monolith with a relational database. This is a deliberate starting point, not a claim that every order system should remain a monolith.

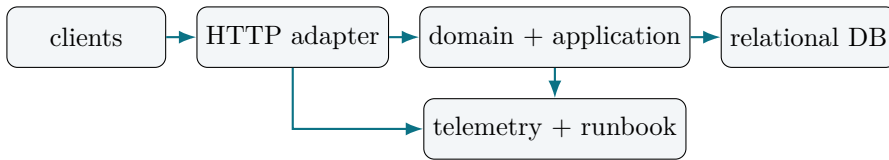
17.2 Requirements and boundaries

The initial slice contains five obligations:

1. validate positive quantities, positive integer prices, and a non-empty customer ID;
2. calculate money exactly in integer cents;
3. persist one accepted order and its items atomically;
4. make retried creation idempotent through a unique key and stored result;
5. expose health and operational signals without exposing personal data.

The system explicitly does not include payment capture, inventory reservation, or international tax law. Excluding those features is a safety decision: their contracts and failure modes have not yet been designed.

17.3 Architecture



The reference repository separates the domain calculation from the standard library HTTP shell. The SQL schema repeats critical domain constraints. The container runs as an unprivileged user with a read-only filesystem and a healthcheck. These are modest controls, but each is visible and testable.

17.4 API contract

An illustrative request is:

Listing 17.1: The body is a data contract, not a Python object.

```

1 POST /orders
2 Idempotency-Key: 8c4a...
3 Content-Type: application/json
4
5 {"customer_id":"customer-1",
6  "items":[{"sku":"book","quantity":2,"unit_price_cents":1250}]}
```

On success, return a stable order identifier, status, and exact money values. For a malformed request, return a client error without writing. For a reused key with a different payload, reject the conflict rather than silently choosing one request. For an unknown outcome after a timeout, let the caller retry with the key and retrieve the stored result.

17.5 Reviewing the implementation

The Python domain module is short enough to inspect line by line:

Listing 17.2: The domain calculation has explicit validation.

```

1 """Small, dependency-free domain model used in the textbook."""
2 from __future__ import annotations
3
4 from dataclasses import dataclass
5 from decimal import Decimal, ROUND_HALF_UP
6 from uuid import uuid4
7
8
9 class ValidationError(ValueError):
```

```

10     """Raised when a request violates a domain invariant."""
11
12
13 @dataclass(frozen=True)
14 class LineItem:
15     sku: str
16     quantity: int
17     unit_price_cents: int
18
19
20 def _positive_int(value: object, field: str) -> int:
21     if isinstance(value, bool) or not isinstance(value, int) or
22     value <= 0:
23         raise ValidationError(f"{field} must be a positive integer
24                                ")
25     return value
26
27
28 def total_cents(items: list[dict[str, object]] | list[LineItem])
29     -> int:
30     """Return an exact integer-money total; floating point is not
31        used."""
32     if not items:
33         raise ValidationError("an order needs at least one line
34                                item")
35     total = 0
36     for item in items:
37         if isinstance(item, LineItem):
38             quantity = item.quantity
39             price = item.unit_price_cents
40             sku = item.sku
41         else:
42             quantity = item.get("quantity")
43             price = item.get("unit_price_cents")
44             sku = item.get("sku")
45             quantity = _positive_int(quantity, "quantity")
46             price = _positive_int(price, "unit_price_cents")
47             if not isinstance(sku, str) or not sku.strip() or len(sku)
48             > 64:
49                 raise ValidationError("sku must be a non-empty string
50                                        of at most 64 characters")
51             total += quantity * price
52     return total
53
54
55 def tax_cents(subtotal_cents: int, rate: str = "0.25") -> int:

```

```
49     """Calculate a tax amount with decimal arithmetic and half-up  
50 rounding."""  
51     _positive_int(subtotal_cents, "subtotal_cents")  
52     value = (Decimal(subtotal_cents) * Decimal(rate)).quantize(  
53         Decimal("1"), rounding=ROUND_HALF_UP)  
54     return int(value)  
55  
56 def create_order(customer_id: str, items: list[dict[str, object]])  
57     -> dict[str, object]:  
58     if not isinstance(customer_id, str) or not customer_id.strip():  
59         :  
60         raise ValidationError("customer_id is required")  
61     subtotal = total_cents(items)  
62     return {  
63         "id": str(uuid4()),  
64         "customer_id": customer_id,  
65         "items": items,  
66         "subtotal_cents": subtotal,  
67         "total_cents": subtotal + tax_cents(subtotal),  
68         "status": "accepted",  
69     }
```

The code deliberately uses integer cents and a decimal library only for the illustrative tax calculation. In a real financial system, tax policy, rounding jurisdiction, currency, refunds, and audit requirements need a larger domain model. The example is not a payment ledger.

The SQL schema is an independent guard. The test runner executes it in an in-memory SQLite database and verifies that the view agrees with the inserted line item. A production database choice changes locking, migrations, backups, and operational procedures; the SQL illustrates principles rather than a vendor recommendation.

17.6 Failure analysis

Failure	Detection	Response
Client retries after lost response	Idempotency-key lookup and duplicate metric	Return stored result; alert on unusual conflict rate.
Database unavailable	Error ratio, dependency latency, health check	Fail closed, preserve request identity, retry with bounded backoff.
Invalid quantity import	Database constraint and validation error	Reject transaction; record safe diagnostic.
Slow dependency added later	Trace and p95/p99 latency	Deadline, isolation, capacity review, or asynchronous workflow.
Secret in log	Secret scanning and log review	Revoke, scrub, investigate access, add regression check.

Each row names a detection and a response. “Add monitoring” would be too vague; the signal has a source and an action.

17.7 What would change at larger scale

At higher traffic, the team might add a connection pool, read replicas, queue workers, partitioning, or independent services. Each change creates new questions: which data is authoritative, how are duplicates reconciled, how are versions coordinated, and how are operators alerted? The existing boundary makes these questions easier to isolate but does not answer them automatically.

Exercise. Extend Ledgerline with a payment authorization workflow. Add states for unknown outcomes, a reconciliation job, an outbox event, and an audit record. Explain why your design does not promise exactly-once effects across the payment provider.

17.8 Chapter summary

A small system can demonstrate serious engineering: explicit scope, exact money, database invariants, idempotency, operational signals, unprivileged deployment, and failure-oriented review. Scaling the system means revisiting constraints, not replacing reasoning with more infrastructure.

Chapter 18

Capstone: from requirement to operated release

18.1 The assignment

Design and implement a service that imports a customer order file, validates it, stores accepted orders, exposes a query endpoint, and produces an operational report. You may use the repository's Ledgerline examples as a starting point, but the capstone should make its own decisions explicit.

18.2 Required deliverables

1. a one-page problem statement with stakeholders, non-goals, and risks;
2. an API and data specification with invalid examples;
3. an architecture diagram and at least two ADRs;
4. a working implementation with unit, component, and migration tests;
5. a threat model and security verification checklist;
6. a container image and CI workflow;
7. an SLO, dashboard or query plan, and recovery runbook;
8. a migration plan for one schema change;
9. an incident scenario and blameless postmortem;
10. a short reflection on accessibility, internationalization, sustainability, and AI-assisted development.

18.3 A release checklist

Done	Check	Evidence
	Behaviour and non-goals are clear	reviewed requirements
	Invalid input is bounded and rejected	tests and validation code
	Authorization is tested on allowed and denied paths	security tests
	Data invariants exist at the right boundary	schema and migration checks
	Retries and unknown outcomes are specified	contract and state model
	Artifact is traceable to source and checks	CI record
	Deployment is observable and reversible	runbook and rollout plan
	Backups or exports can be restored	restore evidence
	Accessibility and locale paths are tested	manual or automated evidence
	Dependencies and licenses are reviewed	inventory and policy

18.4 How to present the design

Begin with the user-visible outcome. Show one complete path, then one failure path. State assumptions before diagrams. Explain why the design is sufficient for the current constraints and what would force a different architecture. Demonstrate a test failure or simulated incident rather than claiming that a feature is “production ready” without evidence.

18.5 Final perspective

The central habit of software engineering is to turn invisible assumptions into visible, testable, revisable decisions. Reliable systems are not produced by one perfect design. They emerge from boundaries that fit the problem, feedback that arrives before consequences become large, and teams that treat operation and maintenance as part of building.

Build the smallest system that satisfies the important constraints, but make its constraints explicit enough that a future team can safely discover when “smallest” is no longer sufficient.

Appendix A

Repository and tool guide

A.1 Quick start

From the repository root:

```
1 make test
2 make pdf
```

The first command runs standard-library checks. It compiles the C++ example with warnings enabled, executes the TypeScript file with Node.js, validates SQL with SQLite, and runs the shell check in a mocked mode. The second command runs LaTeX, BibTeX, MakeIndex, and the final PDF build.

A.2 Example inventory

Path	Purpose
<code>examples/python/ledgerline.py</code>	Domain validation and exact money calculation
<code>examples/python/app.py</code>	Dependency-free health endpoint
<code>examples/typescript/order.ts</code>	Typed client-side calculation
<code>examples/java/RetryPolicy.java</code>	Bounded exponential backoff
<code>examples/cpp/latency_bucket.cpp</code>	Percentile selection over a sample
<code>examples/sql/schema.sql</code>	Relational constraints and an aggregate view
<code>examples/shell/healthcheck.sh</code>	Fail-fast health probe
<code>Dockerfile</code> and <code>compose.yaml</code>	Minimal container and local service orchestration

The examples are intentionally small. A small example can expose a rule without requiring a framework, cloud account, or generated boilerplate. Production use would require authentication, persistence lifecycle, request parsing, rate limits, structured telemetry, deployment policy, and a threat model.

A.3 Validation policy

The repository distinguishes “verified here” from “instructions for another environment.” The Python, SQL, shell, C++, and Node checks are run by the script `scripts/verify_examples.py`. The Java source is dependency-free and includes a compile command; this build environment has a Java runtime but not a compiler, so the CI or a local JDK should compile it. The Docker setup is inspected as source; Docker is optional in the current environment.

This distinction is important. A source file cannot be described as compiled merely because it looks syntactically familiar. Record the toolchain and the exact command used for verification.

A.4 Updating the manuscript

Edit chapter files rather than generated build files. Run ‘make pdf’, render the PDF pages, inspect the title page, contents, code blocks, tables, and diagrams, then run ‘make test’. If references change, rebuild BibTeX and the index. Keep claims about versioned tools tied to a dated reference or a version range.

Appendix B

Cumulative exercises

B.1 Foundations

1. Write a system boundary for a bicycle rental service. Name two actors outside the boundary and three invariants inside it.
2. Convert three vague feature requests into requirements with measurable acceptance criteria.
3. Draw a context diagram for a service that sends notifications through a third-party provider.
4. Identify one stakeholder whose risk is not represented by the product requester.

B.2 Design and data

5. Decompose a document-import service around changing decisions and state invariants.
6. Design an API for cursor pagination. Explain cursor stability under inserts.
7. Model a subscription lifecycle as a state machine and list illegal transitions.
8. Write a schema constraint that prevents a child row from referring to a deleted parent.
9. Design an expand-and-contract migration for renaming a public field.

B.3 Concurrency and architecture

10. Find a lost-update interleaving in a stock reservation operation and repair it.
11. Specify the retry and idempotency behaviour of a message consumer.
12. Compare a modular monolith and two services for a team of three engineers.
13. Write an ADR for choosing a relational database over a document store.

14. State one architecture fitness function for each of three boundaries.

B.4 Quality and operation

15. Build a test matrix for authorization and malformed input.
16. Write a property that should hold for exact integer-money totals.
17. Create an SLO with a defined measurement boundary and monthly error budget.
18. Design a runbook for a growing queue and name the safe first action.
19. Perform a postmortem for an accidentally deleted database column.

B.5 Security and human impact

20. Threat-model a password-reset endpoint and include token replay.
21. Identify personal data in logs, traces, backups, and analytics for an order system.
22. Test a critical flow with keyboard-only navigation and a screen reader.
23. List locale assumptions in a billing statement and replace them with explicit data.
24. Review an AI-generated patch for invented APIs, insecure defaults, and license risk.

B.6 Capstone extensions

25. Add an outbox and an idempotent event consumer to Ledgerline.
26. Add a replayable migration with reconciliation and a failure-injection test.
27. Produce a canary deployment plan with a rollback threshold.
28. Measure the service under a stated workload and explain the bottleneck.
29. Write a one-page architecture review that names what you would not build yet.

Appendix C

Selected solutions and reasoning

C.1 Foundations

1. A bicycle rental boundary can include reservation, payment authorization, and lock commands. The rider, payment provider, and physical lock are outside it. Invariants include one active rental per bicycle, a return cannot precede a start, and a charged amount matches the tariff snapshot.
2. “Fast” becomes a journey, a percentile, a window, and a threshold. For example: p95 from submit to visible confirmation is below 400 ms for a defined payload at a stated load. Add invalid-input and dependency-failure acceptance cases.
3. A notification diagram should show the caller, notification service, provider, delivery callback, and operator. The provider is a trust and reliability boundary; callbacks need authentication and replay handling.

C.2 Design and data

5. Useful modules for document import are format parsing, validation, canonical domain model, persistence, and reporting. The parser should not decide authorization or write directly to the database.
6. A cursor should encode or reference a stable ordering key and direction. It must not assume that page number remains stable while rows are inserted.
7. A subscription state machine might allow trial -> active -> paused -> cancelled, but disallow cancelled -> active without a new subscription or explicit reactivation policy. The rule belongs in one transition function.
8. A foreign key with ‘ON DELETE RESTRICT’ prevents deletion while children exist; ‘ON DELETE CASCADE’ is appropriate only when child loss is intended.

9. Add the new field, write both fields, read the new field with fallback, backfill, enforce the new invariant, then stop writing and remove the old field after the compatibility window.

C.3 Concurrency and architecture

10. Two reservations can both read stock 1. An atomic update such as ‘UPDATE stock SET available = available - 1 WHERE sku = ? AND available > 0’ followed by checking affected rows prevents the lost update.
11. A consumer acknowledges only after its effect is durable. If it crashes before acknowledgment, the message may repeat; use a unique event ID or idempotent upsert and accept that exactly-once effects are not implied.
12. With three engineers and modest traffic, a modular monolith minimizes operational and coordination cost. A service split becomes justified by distinct ownership, legal isolation, or a measured scaling boundary.
13. An ADR should name workload, transactions, operational skill, migration cost, and alternatives. “SQL is standard” is not enough evidence.
14. A dependency rule, migration coexistence test, and tenant-isolation test are three possible fitness functions.

C.4 Quality and operation

15. Cover authenticated allowed access, authenticated denied access, unauthenticated access, malformed identifiers, oversized input, and dependency failure. State which combinations are intentionally omitted.
16. For every valid item list, total equals the sum of positive integer quantity-times-price products and remains within the chosen numeric bound.
17. An SLO must identify eligible requests, success definition, latency boundary, window, and excluded maintenance if any. Its budget is the complement of the target over the eligible measurement population.
18. First stop or rate-limit intake if backlog threatens data freshness, then inspect consumer errors, dependency latency, and poison messages before increasing concurrency.
19. A postmortem should preserve evidence, identify the deletion path and missing guard, restore from a verified backup, and add a migration check plus permission or review control. Blaming the operator is not a control.

C.5 Security and human impact

20. Use short-lived, single-purpose reset tokens stored as hashes, bind them to the account and intent, invalidate on use, rate-limit attempts, avoid account enumeration, and log safe audit events.

21. Remove secrets and unnecessary personal data from telemetry; define access, retention, deletion, and break-glass review for remaining records.
22. Check semantic controls, focus order, visible focus, labels, error announcements, and completion without a pointer device. Confirm with assistive technology rather than relying on a visual inspection.
23. Store canonical instants, currency, locale, and user preferences separately. Localize format at the presentation boundary.
24. Verify every claimed API and dependency, run tests and security checks, inspect data handling, and require a human owner for approval and release.

C.6 Capstone reasoning

25. An outbox row and the order state belong in one local transaction. The consumer uses a unique event ID and records completion. Re-delivery is expected; the external effect must be idempotent or reconciled.
26. A migration should be rerunnable or safely resumable, report rejected rows, compare source and target totals, and stop before destructive work if reconciliation fails.
27. A canary needs a cohort, success indicators, a time window, an owner, and an automatic or human-controlled rollback action. Do not expose a wider audience before telemetry is healthy.
28. Measure a realistic workload and identify the saturated resource. A higher average throughput number without tail and error data is incomplete.
29. The best architecture review names deferred features and the signals that would justify them. Restraint is part of design.

Glossary

API	A defined interface through which software components exchange requests and results.
Availability	The proportion of time a service is usable under a stated measurement definition.
Capacity	The sustainable amount of work a system can handle while meeting its objectives.
Coupling	The degree to which one component depends on another component's details or timing.
Deployment	The act of making a version available in an execution environment.
Idempotency	The property that repeating an operation has the same intended effect as performing it once.
Invariant	A condition that remains true across a specified operation or state transition.
Latency	The elapsed time between an operation's start and completion, measured at a defined boundary.
Observability	The ability to infer internal system state from externally emitted signals.
Recovery point objective	The maximum acceptable amount of data loss measured in time.
Recovery time objective	The maximum acceptable time to restore a service after a failure.
Reliability	The probability that a system performs its required function for a stated period and context.
Service-level objective	A target for a measurable service indicator over a stated window.
Technical debt	A present design or implementation choice whose future cost is knowingly incurred to gain a short-term benefit.
Threat model	A structured description of assets, actors, trust boundaries, threats, and mitigations.

Bibliography

- [1] Beyer, Betsy, Jones, Chris, Petoff, Jennifer, and Murphy, Niall Richard. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.
- [2] Eric Brewer. Cap twelve years later: How the rules have changed. *Computer*, 45(2):23–29, 2012.
- [3] Docker. Compose file reference, 2026.
- [4] Roy T. Fielding, Mark Nottingham, and Julian Reschke. Http semantics. RFC 9110, 2022.
- [5] Martin Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 2 edition, 2018.
- [6] Git Project. Reviewing guidelines, 2026.
- [7] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.
- [8] National Institute of Standards and Technology. Secure software development framework version 1.2: Initial public draft, 2025. Public draft notice; check publication status before treating it as a final standard.
- [9] OpenTelemetry Authors. Opentelemetry specification, 2026. Consult the versioned specification for implementation details.
- [10] OWASP Foundation. Application security verification standard 5.0.0, 2025.
- [11] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.
- [12] Murugiah Souppaya, Karen Scarfone, and Donna Dodson. Secure software development framework (ssdf) version 1.1. Technical Report NIST SP 800-218, National Institute of Standards and Technology, 2022.
- [13] World Wide Web Consortium. Web content accessibility guidelines (wcag) 2.2, 2023.

Index

API, [13](#)

Cohesion, [10](#)

Coupling, [10](#)

fitness function, [22](#)

Logs, [49](#)

Metrics, [49](#)

Software engineering, [1](#)

Traces, [49](#)